Secure Software Design and Development

PNE Report

Submitted to:

Engr Muhammad Ahmad Nawaz

Prepared by:

Maheen Aamir (SP23-BCT-027)

Malik Muhammad Huzaifa (SP23-BCT-029)

Muzamil Ali Akhtar (SP23-BCT-043)

Maryum Masood (SP23-BCT-031)

Department of computer science

(Cyber Security)

1. Project Context

This report covers the security posture of a Flask-based To-Do application. The implementation is structured as follows:

- Auth routes in app/routes/auth.py: /register, /login, /logout (POST)
- Task routes in app/routes/tasks.py: /, /tasks/create, /tasks/<int:task_id>/edit, /tasks/<int:task_id>/delete
- Data model in app/models/task.py: each task has user_id foreign key to User.id

The application's main protection boundary is task ownership enforcement via Task.user_id == current_user.id. All destructive and mutating operations must validate this relationship before proceeding.

2. Assets

The following assets have been identified as requiring protection within this application:

Asset	Where in Code	Why It Matters
User accounts	app/models/user.py (username, password_hash)	Compromise allows impersonation and full task access.
Tasks	app/models/task.py (title, completed, created_at, user_id)	Core user data; confidentiality + integrity are required.
Database	SQLALCHEMY_DATABASE_URI = sqlite:///todo.db in config.py	Single store for all users and tasks.
Session cookies	Flask-Login session after login_user()	Session theft or misuse allows account actions without password re-entry.

3. Threats (Specific to This App)

The following threats have been identified based on the current codebase architecture and exposed attack surface:

Threat	Concrete Scenario in this Codebase	Impact
IDOR	Attacker logs in as user A and tries /tasks/5/edit where task 5 belongs to user B.	Unauthorized read/update/delete of another user's tasks.
CSRF	Victim is logged in; malicious page auto-submits POST to /tasks/<id>/delete or /logout.	Unwanted state changes under victim session.
Clickjacking	Attacker frames app pages and overlays fake UI to trick clicks on delete/logout actions.	User-triggered destructive actions without awareness.
Data Leakage	Debug stack traces (if run with debug=True) or weak authorization exposes internal details/data.	Information disclosure useful for targeted attacks.

4. Security Requirements

The following security controls are required to mitigate the identified threats:

4.1 Object-Level Authorization

- Every task read, update, and delete operation must query with both id and current_user.id.
- Requirement satisfied by _get_user_task_or_404() helper in app/routes/tasks.py.
- No direct object references are permitted without ownership validation.

4.2 CSRF Protection on All Unsafe Actions

- Global Flask-WTF CSRFProtect must be enabled in app/__init__.py.
- All POST forms must include hidden csrf_token fields.
  - Affected forms: login, register, create task, edit task, delete task, logout.
- CSRF tokens must be validated server-side on every state-mutating request.

4.3 Clickjacking Defenses

Responses must include the following HTTP security headers:

- X-Frame-Options: DENY
- Content-Security-Policy: frame-ancestors 'none'

These headers prevent the application from being embedded inside frames on third-party origins, neutralizing UI redressing attacks.

4.4 Session Security

- Keep SESSION_COOKIE_SAMESITE = "Lax" in config.py to mitigate cross-site request forgery via cookie.
- For deployment, enforce SESSION_COOKIE_SECURE = True to restrict cookie transmission to HTTPS.
- Apply production-grade secret management — do not hard-code SECRET_KEY.

4.5 Data Leakage Prevention

- Do not run the application with DEBUG = True in production environments.
- Keep user-facing error messages generic for authentication and task operations.
- Avoid exposing stack traces, internal paths, or SQL errors to end users.

4.6 DevSecOps Verification

- CI pipeline must run: unit tests, CodeQL static analysis, and ZAP baseline DAST scan.
- Build must fail when critical CodeQL vulnerabilities are detected (fail-on-critical gate).
- ZAP baseline report artifact must be reviewed per commit or pull request.

5. DevSecOps Lab Mapping

The following table maps this PNE document to the DevSecOps lifecycle phases, showing where each control is planned, built, verified, gated, and monitored:

Phase	Activity	Details
Plan	PNE	This document identifies assets, threats, and control requirements.
Build	Controls	Implemented in app/routes/tasks.py, app/__init__.py, templates, and config.py.
Verify	CI Pipeline	.github/workflows/ci-cd.yml runs tests + CodeQL + ZAP.
Release Gate	fail-on-critical	Blocks merges/releases on critical SAST alerts.
Monitor	Review Artifacts	Review uploaded ZAP artifact (zap-baseline-report) and code scanning alerts per commit/PR.

6. Risk Prioritization

The identified threats are prioritized based on their impact and likelihood in the current application context:

Threat	Likelihood	Impact	Priority
IDOR	High	High	Critical
CSRF	High	High	Critical
Clickjacking	Medium	Medium	Medium
Data Leakage (Debug Mode)	Medium	High	High

Rationale:

- **IDOR and CSRF** directly affect data integrity and user actions, making them critical.
- **Clickjacking** requires user interaction, lowering its severity.
- **Data leakage** can expose sensitive internal details, aiding further attacks.

7. Assumptions and Constraints

- The application uses **Flask-Login** for authentication and session management.
- SQLite is used as the backend database (single-node, non-distributed).
- No role-based access control (RBAC) is implemented; all users have equal privileges.
- The application is assumed to be deployed behind a standard web server (e.g., Gunicorn/Nginx in production).
- Security controls depend on correct configuration in deployment (e.g., HTTPS, secure cookies).

8. Conclusion

This Protection Needs Elicitation (PNE) identifies critical assets, realistic threat scenarios, and required security controls for the Flask To-Do application.

The analysis confirms that:

- The primary risk lies in **improper access control (IDOR)** and **request integrity (CSRF)**.
- Additional risks include **UI-based attacks (Clickjacking)** and **information disclosure** due to insecure configurations.

By enforcing:

- Object-level authorization
- CSRF protection
- Secure headers
- Proper session handling
- DevSecOps-based verification

the application can achieve a **secure baseline aligned with industry best practices**.

This PNE serves as the foundation for:

- **Threat Modeling (STRIDE)**
- **Risk Assessment (CVSS)**
- **Secure Implementation and Testing**